

Photobox CameraBridge – API-Server (ApiServer.exe) – Handbuch

Stand: 2026-03-25

Zusammenfassung

- Windows-Programm auf Basis von 'net8.0-windows' - stellt eine HTTP/JSON-API bereit - liefert einen MJPEG-LiveView-Stream aus - kommuniziert mit dem Worker per Named Pipe IPC - kann den Worker automatisch starten oder neu anstoßen - stellt Swagger UI / OpenAPI-Dokumentation bereit

1. Überblick

1.1 Hauptaufgaben

1. HTTP-Requests vom Frontend oder Controller annehmen
2. diese in IPC-Commands für den Worker übersetzen
3. Antworten und Fehler als HTTP liefern
4. LiveView-Frames vom Worker pollen und als MJPEG streamen
5. Worker-Erreichbarkeit überwachen
6. API-Dokumentation über Swagger / OpenAPI anzeigen

1.2 Wichtige Bausteine

- **Program.cs**
 - Einstiegspunkt
 - Grundzustand
 - UI-/Tray-Start
- **Program.SingleInstance.cs**
 - Single-Instance- / Replace-Mechanik
- **Program.Hosting.cs**
 - Konfiguration
 - Logging
 - Services
 - Middleware
 - Swagger
 - Start / Stop
- **Api/BridgeApiEndpointMappings.cs**
 - HTTP-Endpunkte
- **Api/ApiRequestDtos.cs**
 - API-spezifische Request-DTOs
- **BridgePipeClient**
 - Named-Pipe-Client zum Worker
- **MjpegStreamer + StreamState**
 - MJPEG-Streaming
- **WorkerHealthMonitor + WorkerHealthState**
 - Health-Monitoring / Reconnect-Logik
- **WorkerProcessManager**
 - Worker-Prozess starten / neu starten

2. Start und Kommandozeilen-Flags

2.1 Unterstützte Flags

Flag	Wirkung	Default
<code>--one-instance true/false</code>	Single-Instance + Replace-Mechanik	true
<code>--one_instance true/false</code>	Alias	–
<code>--on-instance true/false</code>	tolerierter Alias	–
<code>--window_console true/false</code>	Konsole ein- oder ausblenden	true
<code>--tray true/false</code>	Tray aktivieren oder deaktivieren	true
<code>--headless true/false</code>	ohne UI-/Tray-Loop starten	false

Flag	Wirkung	Default
------	---------	---------

Zusätzlich werden CommandLine-Args über `.AddCommandLine(args)` in die Konfiguration übernommen. Damit können Konfigurationswerte direkt überschrieben werden.

2.2 Beispiel für Konfigurations-Override

```
ApiServer.exe --Bridge:Port=8053
```

2.3 Startbeispiele

```
:: Standardstart  
ApiServer.exe
```

```
:: Ohne Single-Instance  
ApiServer.exe --one-instance false
```

```
:: Konsole ausblenden  
ApiServer.exe --window_console false
```

```
:: Komplette headless  
ApiServer.exe --headless true --tray false --window_console false
```

3. Konfiguration (ApiServer_settings.json)

Der API-Server lädt:

- ApiServer_settings.json
- Environment-Variablen mit Prefix PB_
- CommandLine-Args

3.1 Beispielkonfiguration

```
{  
  "Logging": {  
    "LogFile": "ApiServer_log.txt"  
  },  
  "Bridge": {  
    "BindAddress": "0.0.0.0",  
    "Port": 8052,  
    "MjpegPath": "/live.mjpg",  
    "AuthKey": "",  
    "PipeName": "PhotoboxBridge.Cmd"  
  },  
  "Worker": {  
    "ExePath": "C:\\Photobox\\worker.exe",  
    "Args": "--one-instance true --window_console false",  
    "AutoStartOnBoot": true,  
    "AutoStartWhenUnreachable": true,  
    "StartCooldownMs": 8000,  
    "FailThreshold": 3  
  },  
  "Health": {  
    "IntervalMs": 2000,  
    "TimeoutMs": 800  
  }  
}
```

3.2 Environment-Overrides

Durch PB_-Variablen können Werte überschrieben werden:

```
set PB_Bridge__Port=8052
set PB_Bridge__AuthKey=supersecret
set PB_Bridge__PipeName=PhotoboxBridge.Cmd
```

4. Named Pipe IPC zum Worker

4.1 Transport

Die Kommunikation mit dem Worker läuft über eine **Named Pipe** mit:

- 4-Byte Length Prefix (Int32, little-endian)
- danach UTF-8 JSON
- PipeRequest / PipeResponse

4.2 Wichtige Commands

Aus Photobox.Bridge.Shared.Commands:

- status.get
- cameras.list
- camera.select
- camera.refresh
- liveview.start
- liveview.stop
- liveview.fps.get
- liveview.fps.set
- settings.get
- settings.set
- capture
- watchdog.get
- watchdog.set
- frame.wait_next

Wichtig

Der Worker kennt weiterhin **einem** Capture-Command: 'capture'.

Die HTTP-Funktion **Capture** + danach **LiveView** wird im API-Server umgesetzt. Dafür wird nach erfolgreichem Capture zusätzlich 'liveview.start' an den Worker gesendet.

5. Authentifizierung

Wenn Bridge.AuthKey leer ist, ist die Auth-Prüfung deaktiviert.

Wenn Bridge.AuthKey gesetzt ist, sind aktuell **public**:

- GET /
- GET /api/status
- GET {MjpegPath}
 - zum Beispiel /live.mjpg
- GET /favicon.ico
- GET /docs
- GET /docs/index.html
- GET /docs/*
- GET /swagger
- GET /swagger/index.html
- GET /swagger/v1/swagger.json
- GET /openapi/v1.json
- OPTIONS *

Alle anderen Endpunkte benötigen einen Key über:

- X-API-Key: <key>
- Authorization: Bearer <key>

Hinweis

Die Doku-Seiten selbst sind absichtlich öffentlich erreichbar. Die eigentlichen geschützten API-Aufrufe bleiben trotzdem geschützt.

6. Swagger UI und OpenAPI

Verfügbare Routen:

- GET /docs
- GET /docs/index.html
- GET /swagger
 - Redirect auf /docs
- GET /swagger/index.html
 - Redirect auf /docs/index.html
- GET /swagger/v1/swagger.json
- GET /openapi/v1.json
 - Redirect auf /swagger/v1/swagger.json

Damit können Endpunkte sowie Request- und Response-Modelle im Browser angesehen werden.

7. HTTP-API

7.1 Root und Stream

GET / Liefert einen einfachen Textstatus.

GET {MjpegPath} Default: /live.mjpg

Liefert einen MJPEG-Stream als:

```
multipart/x-mixed-replace; boundary=frame
```

7.2 Status und Worker-Health

GET /api/status Kombiniertes API- und Worker-Status.

Enthält unter anderem:

- httpRunning
- liveViewRunning
- selected
- manufacturer
- model
- serial
- httpRunningSinceUtc
- httpUptimeSeconds
- streamRunning
- streamClients
- streamSendingFrames
- framesActive
- framesTotal
- frameAgeMs
- lastFrameUtc
- source
- watchdogEnabled
- workerReachable
- workerLastOkUtc
- workerLastError

GET /api/worker/reachable Nur Health-Snapshot ohne direkten Worker-Call.

GET /api/worker/ping Ping auf den Worker inklusive Latenz in Millisekunden.

POST /api/worker/restart Startet den Worker per `WorkerProcessManager`.

Typische Antwort:

```
{ "ok": true, "error": null }
```

7.3 Kamera

GET /api/cameras Liefert `CameraInfoDto[]`.

POST /api/select?serial=... Wählt eine Kamera anhand der Seriennummer.

POST /api/select?id=... Wählt eine Kamera anhand der ID.

Antwort:

- 200 ok
- 404 not found

POST /api/refresh?timeoutMs=4000 Startet einen Kamera-Refresh über den Worker.

Antwort:

- 200 ok
- Fehler werden über Problem-Responses gemappt, zum Beispiel Busy oder Timeout

7.4 LiveView

POST /api/liveview/start Startet LiveView im Worker.

POST /api/liveview/stop Stoppt LiveView im Worker.

GET /api/liveview/fps Antwort:

```
{ "fps": 10 }
```

POST /api/liveview/fps Mögliche Varianten:

Query:

```
POST /api/liveview/fps?fps=10
```

oder

```
POST /api/liveview/fps?value=10
```

oder JSON-Body:

```
{ "fps": 10 }
```

7.5 Settings

GET /api/settings Liefert `CameraSettingsDto` mit aktuellen Werten und Optionslisten:

- iso
- shutter
- whiteBalance
- isoOptions
- shutterOptions
- whiteBalanceOptions

POST /api/settings Partial Update, Body:

```
{
  "iso": "100",
  "shutter": "1/125",
  "whiteBalance": "Auto"
}
```

Nur diese drei Felder werden vom API-Server an `settings.set` weitergereicht.

7.6 Capture

POST /api/capture Body basiert auf `CaptureRequestDto` und wird im API-Server über `CaptureApiRequestDto` ergänzt.

Wichtige Felder:

- mode: "file" oder "jpeg"
- overwrite
- fileName
- path
- applySettings
- resetAfterShoot
- iso
- shutter
- whiteBalance
- aperture
- exposure

Zusätzliche **API-seitige** LiveView-Flags:

- startLiveViewAfterCapture
- Alias im Body:
 - restartLiveViewAfterCapture
 - captureAndLiveView
 - capturePlusLiveView

Außerdem per Query:

```
POST /api/capture?startLiveViewAfterCapture=true
```

oder

```
POST /api/capture?liveview=true
```

```
{ "ok": true, "file": "C:\\Pfad\\bild.jpg" }
```

Antwort bei mode="file"

Antwort bei mode="jpeg" image/jpeg als Binärantwort.

Verhalten mit LiveView-Flag Wenn das LiveView-Flag gesetzt ist, führt der API-Server nach erfolgreichem Capture zusätzlich `liveview.start` aus.

POST /api/capture-liveview Komfort-Endpunkt.

Verhalten:

1. Capture ausführen
2. danach automatisch `liveview.start`

Der Request-Body entspricht ansonsten `POST /api/capture`.

7.7 Watchdog

GET /api/watchdog Liefert:

```
{ "enabled": true }
```

POST /api/watchdog?enabled=true Setzt den Watchdog im Worker.

8. Fehlerbehandlung

Der API-Server mappt bekannte Worker- und Pipe-Fehler auf HTTP-Problem-Responses.

Typische Mappings:

- device_busy → 409
- no_camera → 404
- cannot_focus → 422
- timeout / refresh_timeout → 504
- unbekannt → 500

Bei Capture-Fehlern wird title = "Capture failed" verwendet.

9. Worker-Autostart und Health-Monitor

9.1 AutoStartOnBoot

Wenn Worker.AutoStartOnBoot = true, versucht der API-Server beim Start den Worker anzustoßen.

9.2 AutoStartWhenUnreachable

Der WorkerHealthMonitor prüft regelmäßig per status.get, ob der Worker erreichbar ist.

Wenn zu viele Fehler in Folge auftreten und AutoStartWhenUnreachable = true gesetzt ist, startet der API-Server den Worker neu oder erneut.

Wichtige Werte:

- Health.IntervalMs
- Health.TimeoutMs
- Worker.FailThreshold
- Worker.StartCooldownMs

10. CORS, Static Files, Sicherheit

- CORS: AllowAnyOrigin / AllowAnyHeader / AllowAnyMethod
- Static Files: wwwroot wird unter / ausgeliefert
- Auth: mit Bridge.AuthKey aktivieren

Checkliste

- 'AuthKey' im LAN / WLAN setzen - 'BindAddress' nur so offen konfigurieren, wie nötig - optional Reverse Proxy / TLS davor setzen

11. Quick-Start

1. ApiServer_settings.json neben ApiServer.exe ablegen
2. Bridge.PipeName passend zum Worker setzen
3. Worker.ExePath korrekt setzen
4. ApiServer.exe starten
5. testen:
 - GET /api/status
 - GET /docs
 - GET /live.mjpg
6. Kamera bedienen:
 - POST /api/refresh